

Regularization to Avoid Overfitting

What is regularization?

model w

$$H(w, x) \sim y$$

Cost function = Loss(y, w) + Regularization term

We have L1 regularization:

$$L1: \|w\|_1 = \sum_{j=1}^m |w_j|$$

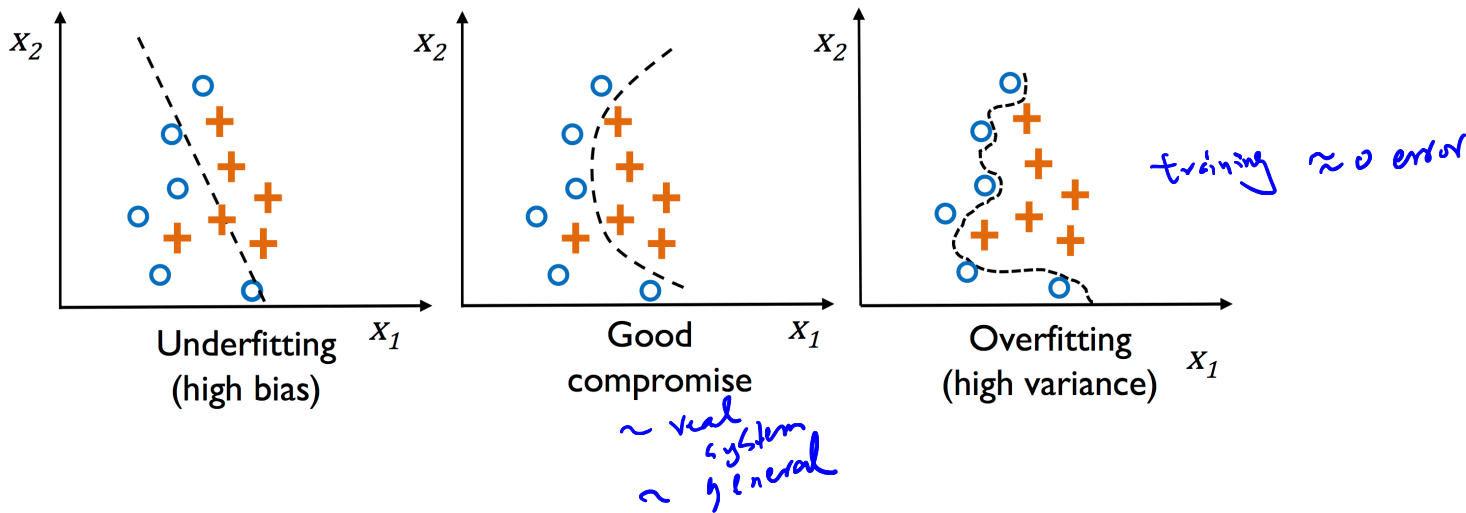
We have L2 regularization:

$$L2: \|w\|_2^2 = \sum_{j=1}^m w_j^2$$

Cost function = Loss(y, w) + $\frac{\lambda}{2m}$ Regularization term

Why we need regularization?

1. Avoid overfitting
2. Avoid NaN for extreme weight (deep neural networks)

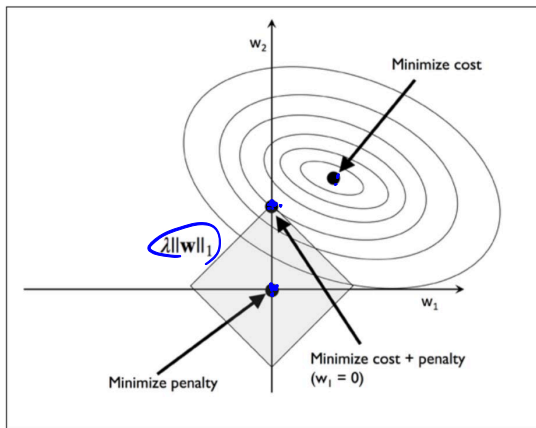


An Intuitive view of regularization penalty

$$\Theta_j^2$$

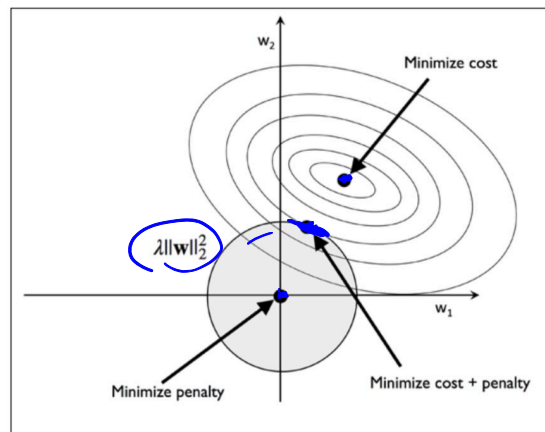
L1 regularization can reduce some features.

$$\sum |w_i|$$



L1

L2 regularization has a smooth curve.

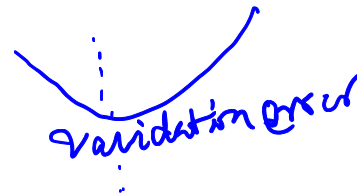


L2

Ridge
Regularization

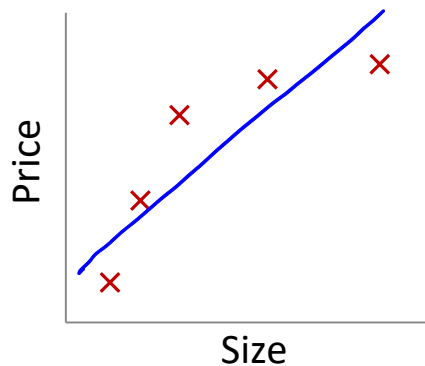
Regularization overfitting problem

1. The problem of overfitting
2. Cost function
3. Regularization for linear regression ✓
4. Regularization for logistic regression ✓
5. Regularization for deep learning ✓



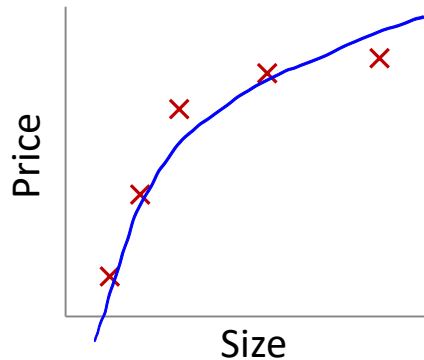
✓
dropout
early stop
cross validation

Example: Linear regression (housing prices)

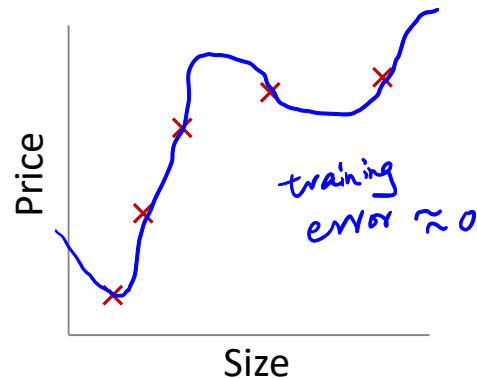


$$\theta_0 + \theta_1 x$$

underfitting



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



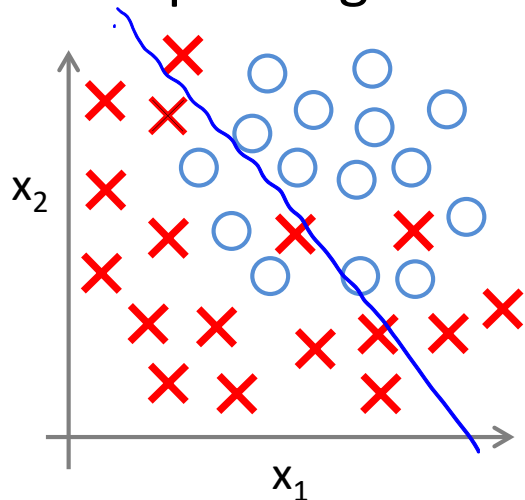
$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

training error ≈ 0
overfitting

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

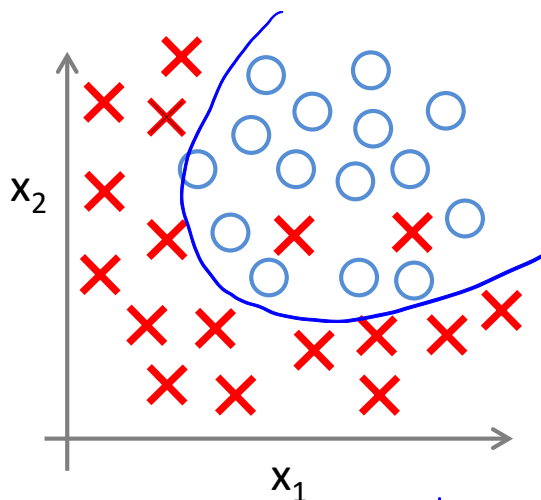
test dataset does not perform well.

Example: Logistic regression



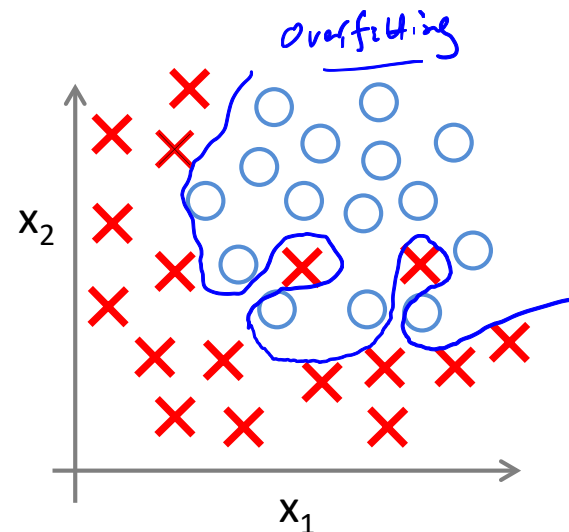
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

(g = sigmoid function)



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

*Quadratic model
(Simple model)*



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

Addressing overfitting:

x_1 = size of house

x_2 = no. of bedrooms

x_3 = no. of floors

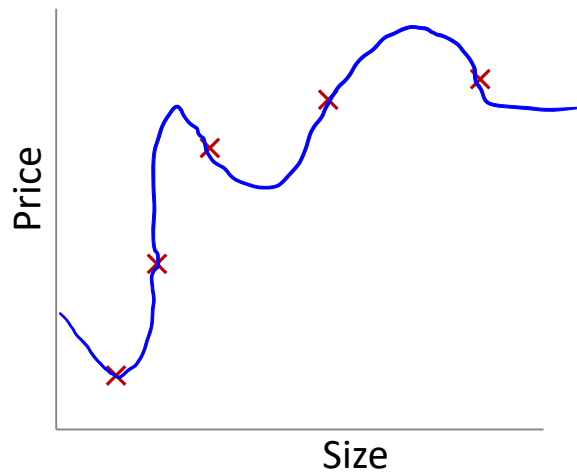
x_4 = age of house

x_5 = average income in neighborhood

x_6 = kitchen size

$\vdots$

x_{100}



too many features

→ limit feature space (size)

PCA

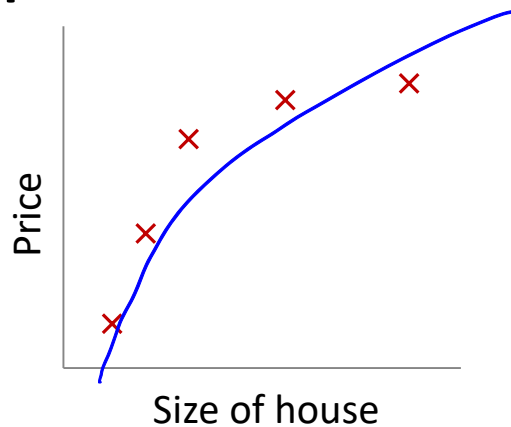
reduce the dimension

Addressing overfitting:

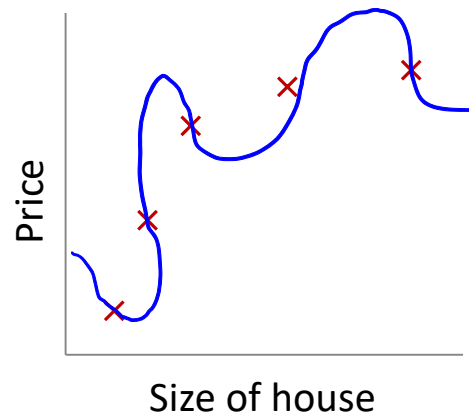
Options:

1. Reduce number of features.
 - Manually select which features to keep.
 - Model selection algorithm (later in course). *PCA*
2. Regularization.
 - Keep all the features, but reduce magnitude/values of parameters θ_j . *θ_j small*
 - Works well when we have a lot of features, each of which contributes a bit to predicting y .

Intuition



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Suppose we penalize and make θ_3, θ_4 really small.

Cost = $\min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 1000 \theta_3^2 + 1000 \theta_4^2$

minimize

loss

force $|\theta_3|, |\theta_4|$ small

Add terms

Regularization

1. The problem of overfitting
- 2. Cost function**
3. Regularization for linear regression
4. Regularization for logistic regression
5. Regularization for deep learning

Regularization.

Small values for parameters $\theta_0, \theta_1, \dots, \theta_n$

- “Simpler” hypothesis
- Less prone to overfitting

Housing:

- Features: $x_1, x_2, \dots, x_{100}$
- Parameters: $\theta_0, \theta_1, \theta_2, \dots, \theta_{100}$

$$J(\theta) = \frac{1}{2m} \left[\underbrace{\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{loss}} + \underbrace{\lambda \sum_{j=1}^n \theta_j^2}_{\text{regularization term, small}} \right]$$

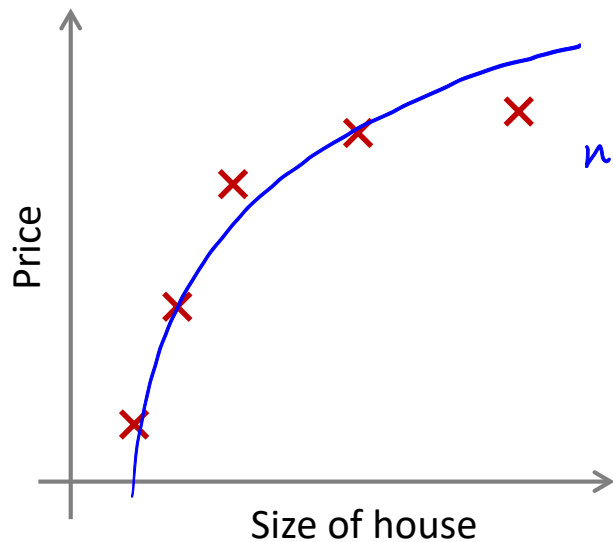
Regularization.

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$\min_{\theta} J(\theta)$$

λ is large

$\lambda = 0$ no regularization

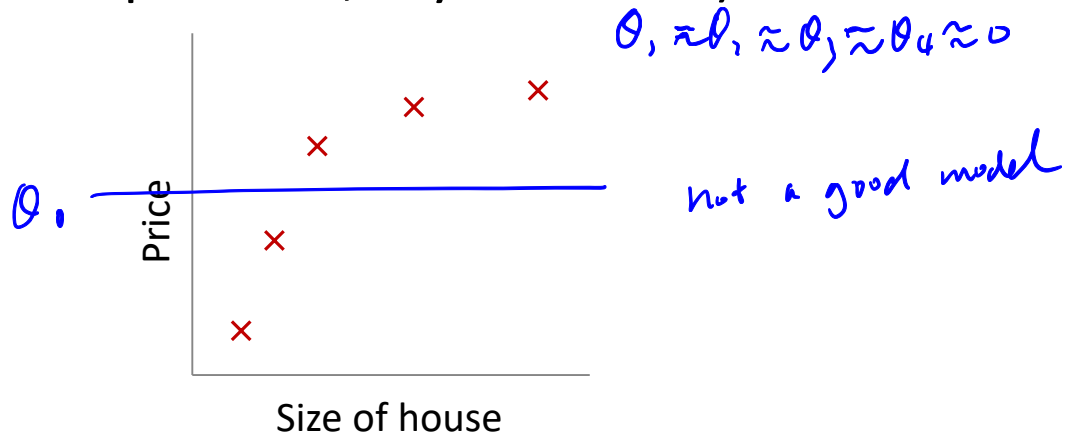


no overfitting

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

What if λ is set to an extremely large value (perhaps far too large for our problem, say $\lambda = 10^{10}$)?



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Regularization

1. The problem of overfitting
2. Cost function
- 3. Regularization for linear regression**
4. Regularization for logistic regression
5. Regularization for deep learning

Regularized linear regression

Loss function

$$J(\theta) = \frac{1}{2m} \left[\underbrace{\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{Loss}} + \underbrace{\lambda \sum_{j=1}^n \theta_j^2}_{\text{Regularization}} \right]$$

$\min_{\theta} J(\theta)$

Gradient method

Update θ_j

$$\frac{\partial J(\theta)}{\partial \theta_j}$$

~~θ_0~~

Gradient descent

$$\underbrace{\theta_0 x_0 + \theta_1 x_1 + \dots + \theta_j x_j}_{\text{bias}}$$

Repeat {

$$\theta_0 := \theta_0 - \underbrace{\alpha}_{\text{learning rate}} \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \quad \frac{\partial J(\theta)}{\partial \theta_0}$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} - \underbrace{\frac{\lambda}{m} \theta_j}_{\text{regularization}} \right] \leftarrow \frac{\partial J(\theta)}{\partial \theta_j}$$

$(j = \text{red X}, 1, 2, 3, \dots, n)$

}

$$\theta_j := \theta_j \left(\underbrace{1 - \alpha \frac{\lambda}{m}}_{\text{update}} \right) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$0 < 1 - 2 \frac{\lambda}{m} < 1$$

≈ 0.99

update

Normal equation

$$X = \begin{bmatrix} (x^{(1)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix}$$

sample

$$y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

$\frac{\begin{matrix} x_0^{(1)} & x_1^{(1)} & \dots & x_j^{(1)} \end{matrix}}{x^{(1)}}$
 $\vdots$
 $\frac{x^{(m)}}{\phantom{x^{(m)}}}$

$$\min_{\theta} J(\theta)$$

$$\theta = (X^T X)^{-1} X^T y$$

Linear regression

multiple features

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_j^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & & x_j^{(2)} \\ \vdots & & & & \vdots \\ 1 & x_1^{(m)} & & & x_j^{(m)} \end{bmatrix}$$

Non-invertibility (optional/advanced).

Suppose $m \leq n$,
(#examples) (#features)

$$\theta = (\underbrace{X^T X}_{\text{not invertible}})^{-1} X^T y$$

(condition number bad)

If $\lambda > 0$,

$$\theta = \left(X^T X + \underbrace{\lambda \begin{bmatrix} 0 & & & \\ & 1 & & \\ & & 1 & \\ & & & \ddots \\ & & & & 1 \end{bmatrix}}_{\text{add additional term}} \right)^{-1} X^T y$$

invertible

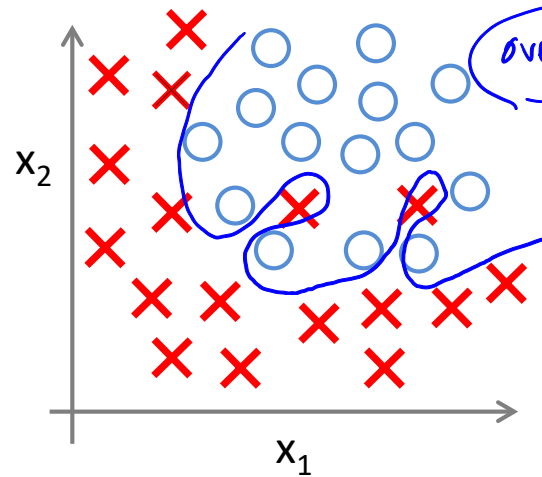
small

Vector form regularization

Regularization

1. The problem of overfitting
2. Cost function
3. Regularization for linear regression
- 4. Regularization for logistic regression**
5. Regularization for deep learning

Regularized logistic regression.



overfitting

Highly nonlinear

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$

Cost function:

$$\min J(\theta) = - \left[\frac{1}{m} \sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

new cost function

Gradient descent

Logistic regression is similar to linear regression

Repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \underbrace{\frac{\lambda}{m} \theta_j}_{\text{regularization}} \right]$$

($j = \text{red X}, 1, 2, 3, \dots, n$)

}

$$\theta_j := \theta_j \underbrace{\left(1 - \alpha \frac{\lambda}{m}\right)}_{\approx 0.99} + \dots$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Advanced optimization

```
function [jVal, gradient] = costFunction(theta)
```

```
    jVal = [code to compute  $J(\theta)$ ] ;
```

$$J(\theta) = \left[-\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

```
    gradient(1) = [code to compute  $\frac{\partial}{\partial \theta_0} J(\theta)$ ] ;
```

θ_0

$$\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

```
    gradient(2) = [code to compute  $\frac{\partial}{\partial \theta_1} J(\theta)$ ] ;
```

θ_1

$$\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)} - \frac{\lambda}{m} \theta_1$$

```
    gradient(3) = [code to compute  $\frac{\partial}{\partial \theta_2} J(\theta)$ ] ;
```

$\vdots$ θ_2

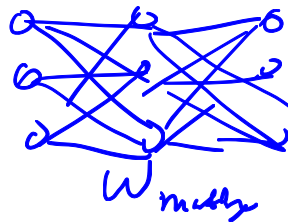
$$\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)} - \frac{\lambda}{m} \theta_2$$

```
    gradient(n+1) = [code to compute  $\frac{\partial}{\partial \theta_n} J(\theta)$ ] ;
```

Regularization

1. The problem of overfitting
2. Cost function
3. Regularization for linear regression
4. Regularization for logistic regression
5. Regularization for deep learning

Deep Learning Regularization via Loss Function



The normal cost function that we want to minimize is:

$\min J(W_1, b_1, \dots, W_L, b_L) = (1/m) * \text{Sum}(L(y(i), y'(i)))$ *Loss function will be different task* *L2 regularization*

samples

The L2 regularization version:

$$J(w, b) = (1/m) * \text{Sum}(L(y(i), y'(i))) + (\lambda/2m) * \text{Sum}(\|W\|^2)$$

loss

$$+ \frac{\lambda}{2m} \sum \|w\|^2$$

regularization

$$\min \sum \|w\|_2^2$$

We stack the matrix as one vector $(mn, 1)$ and then we apply $\text{sqrt}(w_1^2 + w_2^2 + \dots)$

norm

To do back propagation (old way):

$$dw[l] = \text{(from back propagation)}$$

$$w[l] \rightarrow \dots$$

$\Rightarrow$

modify the gradient function

The new way:

$$dw[l] = \text{(from back propagation)} + \lambda/m * w[l]$$

$$\frac{\lambda}{m} * w[l]$$

Deep Learning Regularization via Dropout (trick) for training

- The dropout regularization eliminates some neurons/weights on each iteration based on a probability ($\ll 1$).

Set weights to 0
Randomly select some weights (10%)

- A most common technique to implement dropout is called "Inverted dropout".

- Randomly chosen dropout vector is used for forward and back propagation and is the same for them, but it is different for each iteration (pass) during training. (skip dropout weight)

randomly choose epoch

intuition: not a single weight dominate (overfitting)

- At test time we don't use dropout. If you implement dropout at test time, it would add noise to predictions.

just for training

Other Regularization Methods for Deep Learning

Adversary Learning

GAN

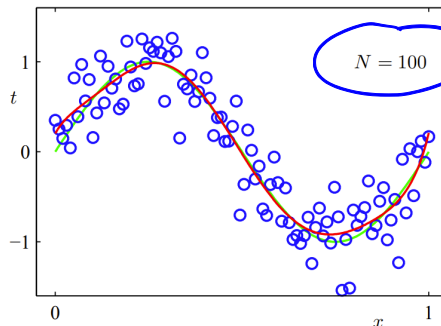
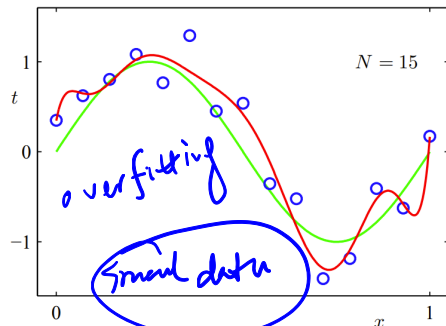
Goodfellow

intuition

- Data Augmentation

generate artificial data

images $\rightarrow$ translation
rotation
reflection
scaling



- Early Stopping

ResNet

